BÖLÜM 4: ASENKRON UFUKLAR — OLAY ODAKLI MIMARI

💣 9 dk • DERS 15

Parça 15: Pratikte Eventual Consistency: Saga Kalıbı vs. İki Aşamalı İşleme (2PC)

Kainatı kilitlemeden çok servisli karmaşık iş süreçlerini başarıyla yürütmek.

> 🔥 **SAHADAN YAŞANMIŞ KRİZ HİKAYESİ**
>
> Bir seyahat portalı tek bir pakette Uçak, Otel ve Kiralık Araç rezervasyonunu birleştirmek istedi. Mimar, mikroservisler arasında dağıtık bir İki Aşamalı İşleme (2-Phase Commit - 2PC) mimarisi kurdu. Yoğun bir günde bir müşteri paket satın almak istedi. 2PC koordinatörü hava yolu veritabanındaki koltuğu ve otel veritabanındaki odayı satır kilidiyle kilitledi. Ardından kiralık araç API'sine istek attı. Araç firmasının API'si ağ sorunu yüzünden tam 42 saniye yanıt vermedi! Bu 42 saniye boyunca o uçak koltuğu ve otel odası kilitli kaldığı için 300 başka müşteri işlem yapamadı. Sonunda araç API'si zaman aşımına uğrayınca koordinatör üç veritabanında birden geri alma (rollback) yapmaya çalışırken kaskad bir kilitlenme yaşandı ve tüm biletleme sistemi dondu.

Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

İki Aşamalı İşleme (2PC) modern bulut mimarilerinde ölçeklenemez. Ağ sınırları boyunca kaynakları kilitleyerek en yavaş ağ bağlantısını tüm şirketin darboğazına dönüştürür. Modern dağıtık mimarinin çözümü **Saga Kalıbı**'dır: Her servisin kendi yerel veritabanını güncelleyip bir sonraki adıma olay fırlattığı yerel işlemler dizisi. Bir adım patlarsa, sistem önceki adımların etkisini tersine çeviren **Telafi Edici İşlemleri (Compensating Transactions)** çalıştırır.

Temel Mimari İlkeler ve Zihinsel Modeller

Koreografi (Choreography) vs. Orkestrasyon (Orchestration)

- **Koreografi (Olay Bazlı):** Her servis olayları dinler ve sıradakini tetikler: `SiparisServisi` -> `OdemeServisi` -> `StokServisi`. 2-3 adım için basittir ama adım sayısı arttıkça kimin neyi başlattığını takip etmek imkansızlaşır.
- **Orkestrasyon (Merkezi Komuta):** Merkezi bir Saga Orkestratörü (Temporal.io, AWS Step Functions) servisleri sırayla yönetir: 'Ödemeyi al. Başarılıysa stoğu düş. Başarısızsa iade yap.' Görselleştirilebilir, izlenebilir ve yönetimi kolaydır.

Telafi Edici İşlemler: Geri Alma (Rollback) Değil, Düzeltme

Veritabanında rollback işlemi hiç yaşanmamış gibi siler. Bir Saga'da ise çalınan zili geri susturamazsınız: Müşteriye SMS gitti, karttan para çekildi. Telafi işlemi yeni bir telafi eylemi yaratır: İptal SMS'i göndermek, iade dekontu kesmek.

Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **Eksik Telafi Yolu:** Saga'nın başarı yolunu harika tasarlayıp 3. adım patladığında önceki adımları telafi edecek otomatik mekanizmayı hiç yazmamak.
- **İdempotent Olmayan Telafi:** İade servisini yeniden denerken idempotency anahtarı kullanmayıp müşteriye iki kat iade yapmak.

- **Dairesel Koreografi:** 8 servisin birbirine olay fırlattığı düzende Servis D'nin Servis B'yi tetiklemesiyle oluşan sonsuz döngüler.

> **🎯 MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR**
>
> - **1. Adım:** İş süreci 3 adımı aşıyorsa veya finansal işlemler içeriyorsa doğrudan Orkestrasyon modelini seç.
> - **2. Adım:** Kod yazmadan önce her adımın 'Başarı' ve 'Başarısızlık Telafisi' senaryosunu kağıda dök.
> - **3. Adım:** Zaman aşımlarını ve yeniden denemeleri yönetmek için Temporal.io gibi hazır bir orkestrasyon motoru kullan.

Mimari Çıkarımlar ve Hile Tablosu

- Mikroservisler arasında asla 2-Phase Commit kullanmayın; dağıtık kilitler sisteminizi felç eder.
- Saga kalıbı ACID atomikliğini iş seviyesinde eventual consistency ve esneklikle takas eder.
- Karmaşık süreçlerde merkezi bir orkestratör, olay koreografisini her zaman mağlup eder.

> **🎙️ UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU**
>
> **Ders Değerlendirmesi ve Ödev:** Uygulamanızdaki çok adımlı bir süreci çizin (örn: sipariş, üyelik). 3. adım patladığında 1. ve 2. adımdaki işlemler otomatik olarak nasıl telafi ediliyor?